



《智能交通监控管理系统》

项目详细设计

V1.0



版本历史

版本/ 状态	作者	参与者	日期	备注
1.0	李艺航	张恒茂、刘雨涵、郑好晴、王子画	2025-08-29	创建



目 录

第一部分 引言	5
一、编写目的	5
二、项目背景	5
三、定义	5
1. 技术类	5
2. 业务类	8
四、参考资料	8
第三部分 总体设计	10
一、技术架构设计	10
二、核心控制流程	12
1. 核心控制流程图	12
2. 核心控制流程说明	12
三、四层架构集成配置	14
1. View 层配置	14
2. Controller 层配置	18
3. Service 层配置	20
第四部分 界面设计	22
一、界面设计	23
1. 登录界面设计	23
2. 主体界面设计	23
3. 实时监控界面设计	24
4. 信息查询界面设计	24
5. 违规查询界面设计	25
6. 流量查询界面设计	25
7. 关于我们界面设计	26
第五部分 单元模块设计	26
一、Model 层设计	26
1. 类图设计	26
2. 类的详细设计描述	28
二、Service 层设计	41
1. 类图设计	41
2. 类的详细设计描述	42



三、Controller 层设计	46
1. 类图设计	46
2. 类的详细设计描述	46
四、View 层设计	52
1. 界面功能	52
2. 类的详细设计描述	52
第六部分 数据库设计	58
一、数据库整体结构图	58
二、数据库表设计	58
1. 用户表 (User)	58
2. 车辆表 (Vehicle)	58
3. 行人表 (Pedestrian)	59
4. 非机动车表 (NonMotorVehicle)	59
5. 道路表 (Road)	59
6. 信号灯表 (TrafficLight)	60
7. 交通流量表 (TrafficFlow)	60
8. 违章事件基表 (ViolationEvent)	61
第七部分 补充设计和说明	61
一、编译运行环境设计	61
1. 基础环境要求	61
2. 核心依赖说明	62
3. 部署步骤	62
4. 验证与问题排查	63
5. 维护说明	63



第一部分 引言

一、编写目的

编写本设计的目的是为了准确阐述智能交通监控系统的具体实现思路和方法，即系统的详细架构和实现逻辑，主要包括程序系统的结构以及各层次中每个程序的设计考虑。预期读者为项目全体成员，包括运行维护和测试人员。

二、项目背景

- 系统名称：智能交通监控系统
- 任务提出者：略
- 开发者：略
- 用户和运行该程序系统的计算中心：略

三、定义

1. 技术类

本系统采用前后端分离的架构。APP 端负责 UI 交互和部分离线逻辑，云端后端提供集中的数据存储、复杂的业务处理和模型分析能力。

1.1 APP 端架构



1.1.1 表现层 (Presentation Layer)

表现层作为用户直接感知的核心交互层，承担着 APP 的用户界面 (UI) 展示与用户操作响应职责，涵盖监控大屏、违章列表等所有屏幕页面的构建与绘制，同时精准处理用户的点击、滑动、输入等操作并触发对应事件；在数据交互上，实现界面控件与底层数据模型的绑定，确保数据变化时 UI 自动更新，还负责管理页面间的跳转、传参及返回逻辑，且能适配不同尺寸、分辨率的移动设备，为用户提供一致且流畅的使用体验。

1.1.2 领域层 (Domain Layer)

领域层是 APP 的核心业务逻辑层，具备独立于任何框架的“纯净”特性，明确规定 APP “要做什么”，以业务用例为核心载体，将“信息查询”“登录”等用户操作或具体业务功能封装为独立用例，同时执行轻量级、与 UI 无关的业务规则校验（如提交表单前校验输入数据有效性），并定义核心业务实体模型，这些实体不包含任何平台相关代码；正因为不依赖表现层和数据层，该层可轻松开展单元测试，保障核心业务逻辑的稳定性与可靠性。

1.1.3 数据层 (Data Layer)

数据层专注于管理 APP 的数据来源，承担“怎么做”来获取数据的职责，其核心是 Repository（仓库）模式，作为单一数据源的门面，Repository 会智能决策数据获取途径（如网络优先或缓存优先），并向上层提供统一数据接口；具体而言，通过远程数据源 (Remote Data Source) 借助 Retrofit (Android) 或 Alamofire (iOS) 等库，以 API 与云端后端通信实现数据的获取和提交，同时利用本地数据源 (Local Data Source) 通过 SQLite、Room (Android) 或 CoreData (iOS) 等本地数据库缓存数据，支持离线查看与快速访问，此外还需完成数据模型转换工作，将 API 的网络数据模型 (DTOs) 和本地数据库的数据模型 (DAOs)，统一转换为领域层可使用的实体模型 (Entities)。



1.2 云端后端架构

1.2.1 API 接口层 (API Layer)

API 接口层是云端后端为 APP 提供服务的统一入口，核心职责是接收、校验并响应 APP 的所有网络请求，保障交互的安全性与规范性；该层会设计清晰的 API 端点用于路由与请求接收，完成 APP 用户的身份认证与权限校验（如限制用户仅能查询自身违章信息），同时负责解析 APP 发送的 JSON 请求，并将后端处理结果封装成 JSON 格式返回；为应对安全风险，还会实施速率限制与安全防护措施，有效防止恶意请求和攻击行为，维护后端服务的稳定运行。

1.2.2 应用服务层 (Application Service Layer)

应用服务层作为云端后端的“大脑”，是实现系统核心、复杂业务逻辑与流程编排的关键层级，专注于处理 APP 端无法完成的重型任务，例如调用模型分析视频流并生成违章记录、对海量数据进行统计与分析等复杂业务处理；在数据操作层面，该层具备事务管理能力，确保涉及多个数据库操作的任务具有原子性，避免数据不一致问题；同时，根据业务需求可能调用用户服务、模型分析服务、消息推送服务等内部微服务，通过协同工作共同完成 APP 的请求，且会落地系统最重要的业务规则，保障整体业务流程的合规性与高效性。

1.2.3 数据持久层 (Data Persistence Layer)

数据持久层承担着云端后端所有业务数据的持久化存储职责，是与数据库交互的核心环节；该层借助 SQLAlchemy 等 ORM 框架，实现业务对象与 MySQL/PostgreSQL 数据库表的映射，为数据操作提供便捷的对象化接口；在具体功能上，不仅提供数据的 CRUD（创建、读取、更新、删除）基础操作，还会针对 APP 的数据请求场景，编写高效的 SQL 查询语句进行复杂查询优化，确保数据存取的高效性与准确性，为上层业务逻辑的运转提供稳定的数据支



撑。

2. 业务类

监控设备终端：安装在道路、路口、停车场等位置的硬件设备，是系统采集数据的基础单元，每个设备具有唯一标识（如设备 ID）。

车辆电子标识：车辆安装的电子标签（类似 RFID），存储车辆唯一身份信息，支持与路侧设备无线通信（如 ETC 扩展功能）。

交通事件记录：系统自动识别或人工上报的交通相关事件，是监控系统的核心管理对象，需关联设备、车辆、时间等信息。

四、参考资料

列出有关的参考资料，如：

- 1) 本项目的经核准的计划任务书或合同、上级机关的批文；
- 2) 属于本项目的其他已发表的文件；

《<智能交通监控管理系统>项目需求说明书》

《<智能交通监控管理系统>项目概要设计》

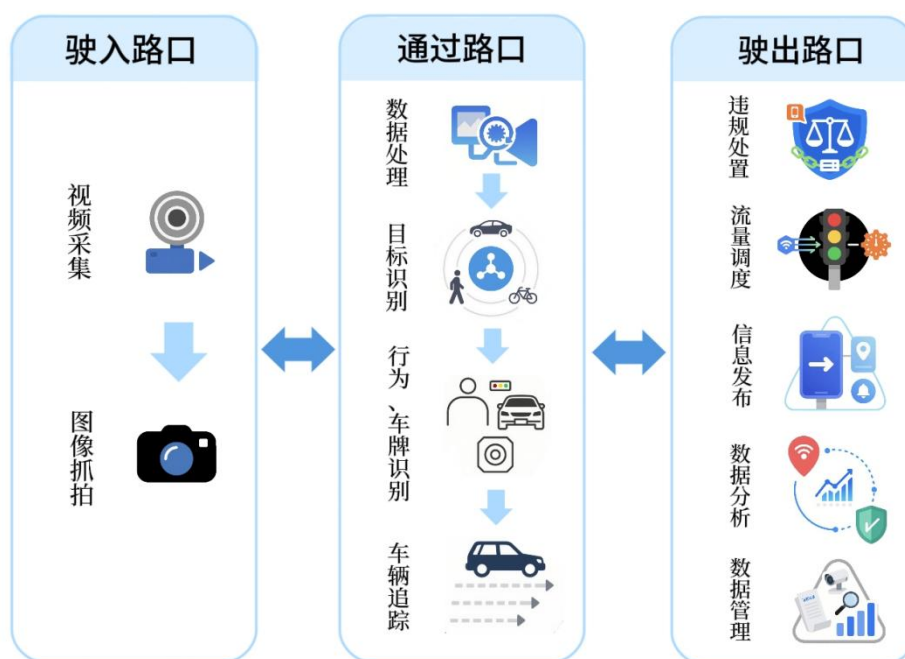
- 3) 本文件中各处引用到的文件资料，包括所要用到的软件开发标准。

《PEP 8》

第二部分 项目概述

基于物联网的智能交通监控系统，依托部署于云端的 IOT 应用，构建车辆、道路设施、监控终端、管理平台、智能传感器、交通信号设备等“物与物”的智慧联动体系。凭借技术创新与模式适配，深度赋能交通管理场景，有效提升交通管控效能、通行效率与服务质量，为城市交通治理、出行服务优化提供数字化支撑，创造显著的社会与经济效益。

系统聚焦路口通行全流程监管，涵盖驶入监测（视频采集、图像抓拍）、通行管控（数据处理、目标识别、行为与车牌识别、车辆追踪）、驶出治理（违规处置、流量调度、信息发布、数据分析、数据管理）三大核心业务环节。具备 7*24 小时安全、高效、无故障运行能力，交通管理及运维人员可便捷完成设备状态监控、交通事件处置、业务流程管理，丰富的分析报表满足交通态势研判、执法证据回溯、优化决策等多元需求，助力打造智慧、有序、畅通的交通生态。





第三部分 总体设计

一、技术架构设计

本系统采用基于 Python 的现代分层架构，通过 Model 层、Service 层、Controller 层和 View 层的清晰划分，构建了一个职责明确、高效协同的开发体系。

在 Model 层，我们通过对象关系映射技术定义数据模型类并编写数据访问对象，深入分析数据模型关系，同时结合数据验证机制进行请求/响应校验，并实现高效的异步数据库操作方法，为整个系统提供坚实的数据基础。

Service 层作为业务核心，负责调用 Model 层进行数据操作，实现各类业务逻辑服务类。该层不仅处理核心业务逻辑和异常情况，还深度集成机器学习服务，将 PyTorch 和 OpenCV 等 AI 能力融入智能交通监控的各项业务场景中，确保业务处理的完整性和事务一致性。

Controller 层则扮演承上启下的关键角色，通过调用 Service 层处理具体业务，编写路由处理函数，基于路由机制接收并验证请求数据，最终返回标准化响应，支持 JSON、XML 等多种数据格式输出。

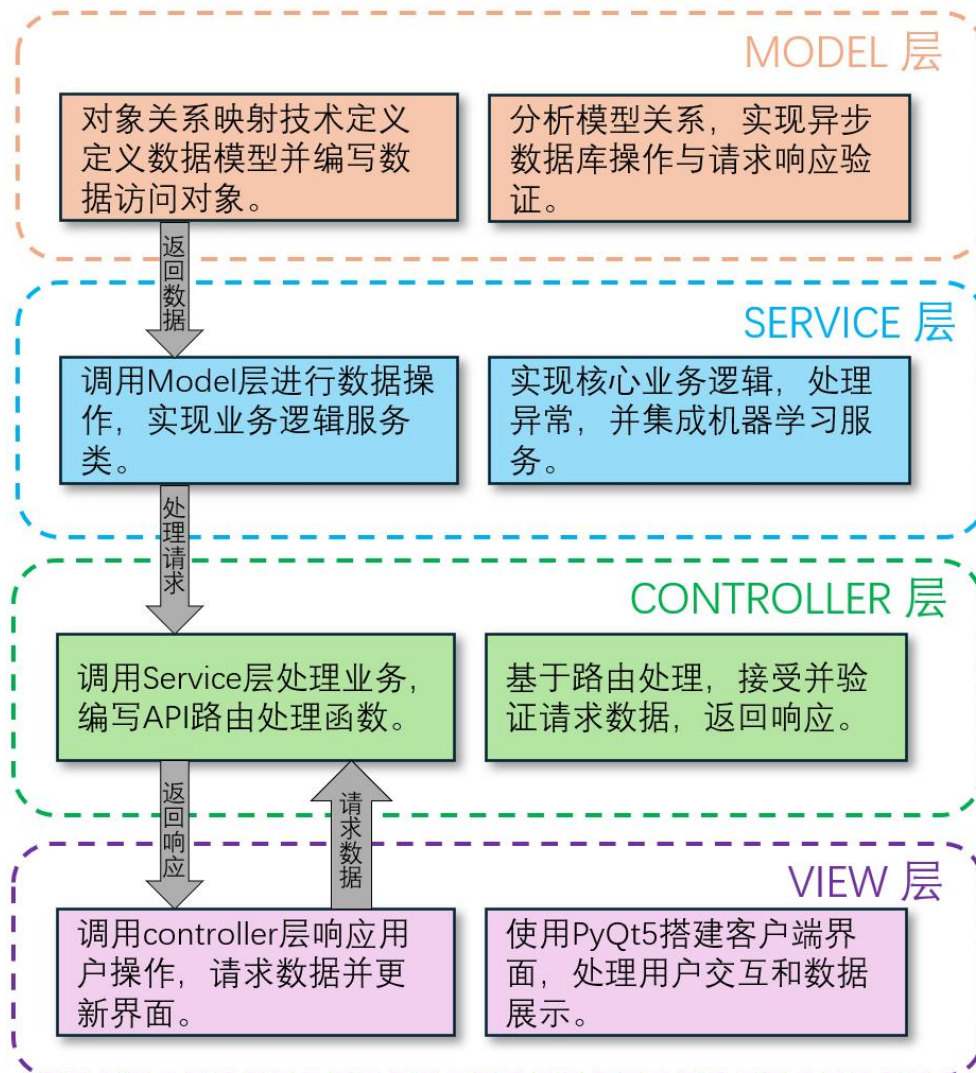
View 层作为用户交互的入口，通过调用 Controller 层响应用户操作，发送数据请求并更新界面显示。利用 PyQt5 搭建的客户端界面提供了丰富的用户交互体验，同时 APP 前端界面确保了对多终端设备的支持，实现了用户交互与数据展示的有效结合。

该架构充分利用 Python 的异步支持与依赖注入特性，确保业务逻辑集中于 Service 层处理，同时使同一份核心代码无需修改即可适应多种数据输出格式，极大简化了前后端交互流程，显著提升了开发效率。结合数据验证机制和 Python 异步生态的高性能特性，本架构



为智能交通监控系统提供了一个职责清晰、高效可靠且易于扩展的技术基础，充分满足了智能交通系统对实时性、可靠性和可扩展性的高标准要求。其开发的结构图如下：

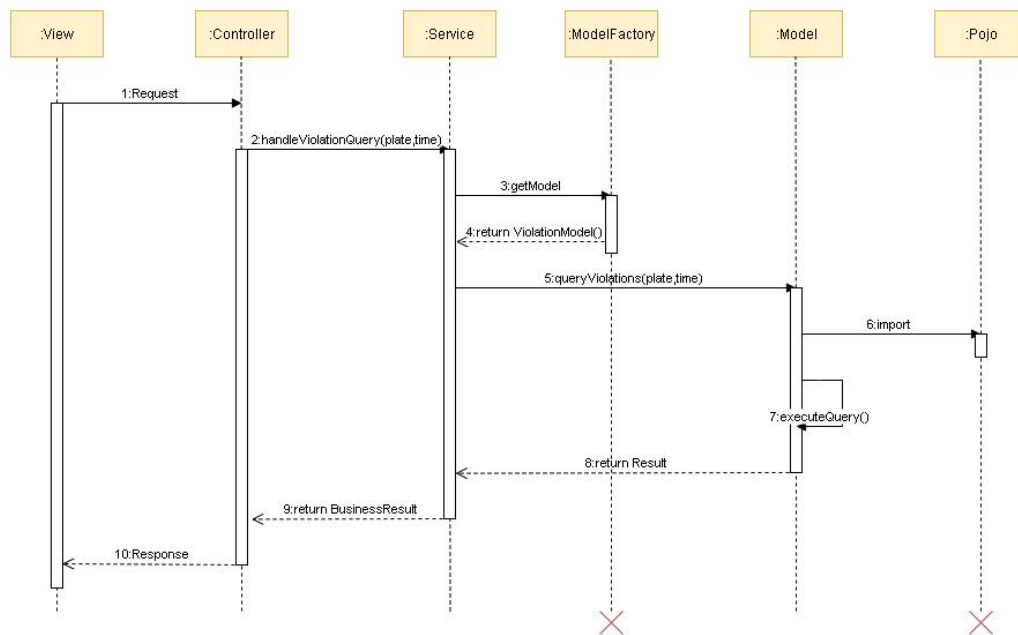
其开发的结构图如下：





二、核心控制流程

1. 核心控制流程图



2. 核心控制流程说明

结合上图，针对 HTTP 请求，对处理流程作如下说明。

编号	消息名称	消息说明
1	交互请求	View 层通过 Qt 组件（按钮/菜单）发起请求，请求通过 Python 代码传递至 Controller 层
2	请求预处理	Controller 层通过 Pydantic 校验参数，调用 JWT 工具验证用户令牌有效性



3	业务方法调用	Controller 层根据请求类型分发至对应 Service 层方法
4	业务逻辑执行	Service 层处理业务规则、执行事务控制、捕获业务异常
5	数据操作调用	Service 层根据业务需求调用 Model 层接口，明确 MySQL 数据操作类型
6	MySQL 数据库操作	Model 层通过 SQLAlchemyORM 将实体类方法转换为 MySQL 语句，执行 MySQL 交互
7	数据库结果返回	MySQL 返回操作结果：查询类请求返回符合条件的数据集，写入/更新类请求返回影响行数
8	数据封装	Model 层将 MySQL 返回的原始数据映射为系统实体类对象，屏蔽 MySQL 字段与代码对象的差异，便于上层调用
9	业务结果返回	Service 层整合数据与业务状态，传递至 Controller 层
10	响应封装与返回	Controller 层将结果封装为 View 层可识别格式，通过信号触发 View 层更新
11	界面渲染	View 层接收信号后更新 Qt 组件，
12	ActionForward.findForward	Action 返回执行结果给新的画面 (Struts-Config.xml 配置)



13	HttpServletResponse	MappingDispatchAction 返回处理结果给浏览器
----	---------------------	----------------------------------

三、四层架构集成配置

1. View 层配置

```
001     class MainWindow(QWidget, Ui_Form):
002         def __init__(self):
003             super().__init__()
004             self.setupUi(self)
005
006             # 获取当前登录用户并显示在界面上
007             self.session_manager = SessionManager()
008             current_user = self.session_manager.get_current_user()
009             if current_user:
010                 self.user_name.setText(current_user)
011
012             # 设置窗口属性
013             self.setWindowFlags(Qt.FramelessWindowHint) # 无边框窗口
014             self.setMinimumSize(1331, 831) # 设置最小尺寸
015
016             # 添加关闭回调属性
017             self.close_callback = None
018
019             # 初始化导航管理器
020             self.nav_manager = NavigationManager(self)
021
022             # 初始化车辆表格管理器
023             self.vehicle_table_manager = VehicleTableManager(self)
024
025             # 初始化违规表格管理器
026             self.violation_table_manager = ViolationTableManager(self)
```



```
027
028         # 初始化时间和天气显示管理器
029         self.weather_time_manager = WeatherTimeDisplayManager(self)
030
031         # 初始化流量表格管理器
032         self.flow_table_manager = FlowTableManager(self)
033
034         # 初始化关于我们页面管理器
035         self.about_manager = AboutManager(self)
036
037         # 初始化视频播放
038         self.video_view = VideoView(self)
039         from controllers.videoController import VideoController
040         self.video_controller = VideoController(self)
041         self.video_view.set_controller(self.video_controller)
042
043         # 设置 left_content 布局
044         self.setup_left_content_layout()
045
046         # 设置 right_content 布局
047         self.setup_right_content_layout()
048
049         # 设置违规查询页面布局
050         self.setup_violation_layout()
051
052         # 设置流量查询页面布局
053         self.setup_flow_layout()
054
055         # 连接顶部按钮功能
056         self.setup_top_buttons()
057
058         # 可以在这里添加其他初始化代码
059         self.setWindowTitle("智能交通管理系统")
060
```



```
061     def setup_top_buttons(self):
062         """
063         设置顶部按钮功能
064         """
065         # 连接关闭按钮 - 关闭主窗口并返回登录界面
066         self.closeAppBtn.clicked.connect(self.close) # 这里保持不变
067
068         # 连接最大化/恢复按钮
069         self.maximizeRestoreAppBtn.clicked.connect(self.toggle_maximize)
070
071         # 连接最小化按钮
072         self.minimizeAppBtn.clicked.connect(self.showMinimized)
073
074     def toggle_maximize(self):
075         """
076         切换窗口最大化/恢复状态
077         """
078         if self.isMaximized():
079             self.showNormal()
080         else:
081             self.showMaximized()
082
083     # main_detect.py (另一种方式)
084     def setup_left_content_layout(self):
085         """
086         设置左侧内容区域布局
087         """
088         # 调用表格管理器设置布局
089         self.vehicle_table_manager.setup_left_content_layout()
090
091         # 单独设置表格最小高度为 80 像素
092         self.vehicle_table_manager.set_table_min_height(150)
093
094     def setup_right_content_layout(self):
```




```
095         """
096         设置右侧内容区域布局
097         """
098         # 调用时间和天气显示管理器设置布局
099         self.weather_time_manager.setup_right_content_layout()
100
101     def setup_violation_layout(self):
102         """
103         设置违规查询页面布局
104         """
105         # 调用违规表格管理器设置布局
106         self.violation_table_manager.setup_violation_layout()
107
108     def setup_flow_layout(self):
109         """
110         设置流量查询页面布局
111         """
112         # 调用流量表格管理器设置布局
113         self.flow_table_manager.setup_flow_layout()
114
115     # def closeEvent(self, event):
116     #     """
117     #     窗口关闭事件, 停止定时器并返回登录界面
118     #     """
119     #     # 停止所有定时器
120     #     self.weather_time_manager.stop_timers()
121     #
122     #     # 调用关闭回调(如果存在)
123     #     if self.close_callback:
124     #         self.close_callback()
125     #
126     #     event.accept()
127
128     def closeEvent(self, event):
```



```
129         """
130         窗口关闭事件, 停止定时器并返回登录界面
131         """
132         # 停止所有定时器
133         self.weather_time_manager.stop_timers()
134
135         # 停止视频播放并关闭数据库连接
136         if hasattr(self, 'video_controller') and self.video_controller:
137             self.video_controller.stop_video()
138             # 如果你的 VideoService 有 close_database_connection 方法, 也调用它
139             if hasattr(self.video_controller.video_service,
140 'close_database_connection'):
141                 self.video_controller.video_service.close_database_connection()
142
143             # 调用关闭回调(如果存在)
144             if self.close_callback:
145                 self.close_callback()
146
147         event.accept()
```

2. Controller 层配置

```
01 class VideoController:
02     def __init__(self, view):
03         self.view = view
04         self.video_service = VideoService()
05
06     def open_video(self):
07         """
08         处理打开视频文件的请求
09         """
10         return self.video_service.open_video_file()
11
12     def play_video(self, file_path):
```



```
13         """
14         处理播放视频的请求
15         """
16         return self.video_service.play_video(file_path)
17
18     def pause_video(self):
19         """
20         处理暂停视频的请求
21         """
22         self.video_service.pause_video()
23
24     def resume_video(self):
25         """
26         处理恢复视频播放的请求
27         """
28         self.video_service.resume_video()
29
30     def stop_video(self):
31         """
32         处理停止视频的请求
33         """
34         self.video_service.stop_video()
35
36     def seek_video(self, position):
37         """
38         处理跳转到指定位置的请求
39         """
40         self.video_service.seek_to_position(position)
41
42     def get_frame(self):
43         """
44         获取当前帧
45         """
46         return self.video_service.get_frame()
```



```
47
48     def process_frame_for_detection(self, frame):
49         """
50         处理帧以进行目标检测
51         """
52         return self.video_service.process_frame_for_detection(frame)
```

3. Service 层配置

```
01     class VideoService:
02         MISSING_THRESHOLD = 5.0 # 5 秒超时
03         def __init__(self):
04             self.video_capture = None
05             self.is_playing = False
06             self.detector = None # 延迟初始化目标检测器
07
08             # self.database_service = DatabaseService() # 数据库服务
09
10             self.database_service = DatabaseService() # 数据库服务, 延迟初始化
11             self.current_video_path = None # 当前视频路径
12
13
14             # 用于目标检测的 polygon mask
15             self.polygon_mask_blue_and_yellow = None
16             self.color_polygons_image = None
17
18             # 不在初始化时创建 polygons
19
20             # 视频开始时间
21             self.video_start_time = None
22
23             # 跟踪对象列表
24             self.tracked_objects = {} # 存储正在跟踪的对象信息
25
```



```
26         # 撞线检测相关
27         self.list_overlapping_blue_polygon = []
28         self.list_overlapping_yellow_polygon = []
29         self.down_count = 0
30         self.up_count = 0
31
32     def _ensure_detector_initialized(self):
33         """
34         确保目标检测器已初始化
35         """
36         if self.detector is None:
37             self.detector = Detector()
38
39     def _ensure_detection_polygons_initialized(self):
40         """
41         确保检测 polygons 已初始化
42         """
43         if self.polygon_mask_blue_and_yellow is None or self.color_polygons_image is None:
44             self._init_detection_polygons()
45
46     def _init_detection_polygons(self):
47         """
48         初始化目标检测用的 polygon 区域
49         """
50         # 根据视频尺寸, 填充一个 polygon, 供撞线计算使用
51         mask_image_temp = np.zeros((1080, 1920), dtype=np.uint8)
52
53         # 初始化 2 个撞线 polygon
54         list_pts_blue = [[204, 305], [227, 431], [605, 522], [1101, 464], [1900, 601], [1902,
55 495], [1125, 379], [604, 437],
56                        [299, 375], [267, 289]]
57         ndarray_pts_blue = np.array(list_pts_blue, np.int32)
58         polygon_blue_value_1 = cv2.fillPoly(mask_image_temp, [ndarray_pts_blue], color=1)
59         polygon_blue_value_1 = polygon_blue_value_1[:, :, np.newaxis]
```



```
60
61         # 填充第二个 polygon
62         mask_image_temp = np.zeros((1080, 1920), dtype=np.uint8)
63         list_pts_yellow = [[181, 305], [207, 442], [603, 544], [1107, 485], [1898, 625],
64 [1893, 701], [1101, 568],
65                             [594, 637], [118, 483], [109, 303]]
66         ndarray_pts_yellow = np.array(list_pts_yellow, np.int32)
67         polygon_yellow_value_2 = cv2.fillPoly(mask_image_temp, [ndarray_pts_yellow],
68 color=2)
69         polygon_yellow_value_2 = polygon_yellow_value_2[:, :, np.newaxis]
70
71         # 撞线检测用 mask, 包含 2 个 polygon, (值范围 0、1、2), 供撞线计算使用
72         self.polygon_mask_blue_and_yellow = polygon_blue_value_1 + polygon_yellow_value_2
73
74         # 缩小尺寸, 1920x1080->960x540
75         self.polygon_mask_blue_and_yellow = cv2.resize(self.polygon_mask_blue_and_yellow,
76 (960, 540))
77
78         # 蓝色盘 b,g,r
79         blue_color_plate = [255, 0, 0]
80
81         # 蓝 polygon 图片
82         blue_image = np.array(polygon_blue_value_1 * blue_color_plate, np.uint8)
83
84         # 黄色盘
85         yellow_color_plate = [0, 255, 255]
86
87         # 黄 polygon 图片
88         yellow_image = np.array(polygon_yellow_value_2 * yellow_color_plate, np.uint8)
89
90         # 彩色图片(值范围 0-255)
91         self.color_polygons_image = blue_image + yellow_image
92
93         # 缩小尺寸, 1920x1080->960x540
94         self.color_polygons_image = cv2.resize(self.color_polygons_image, (960, 540))
```

第四部分 界面设计

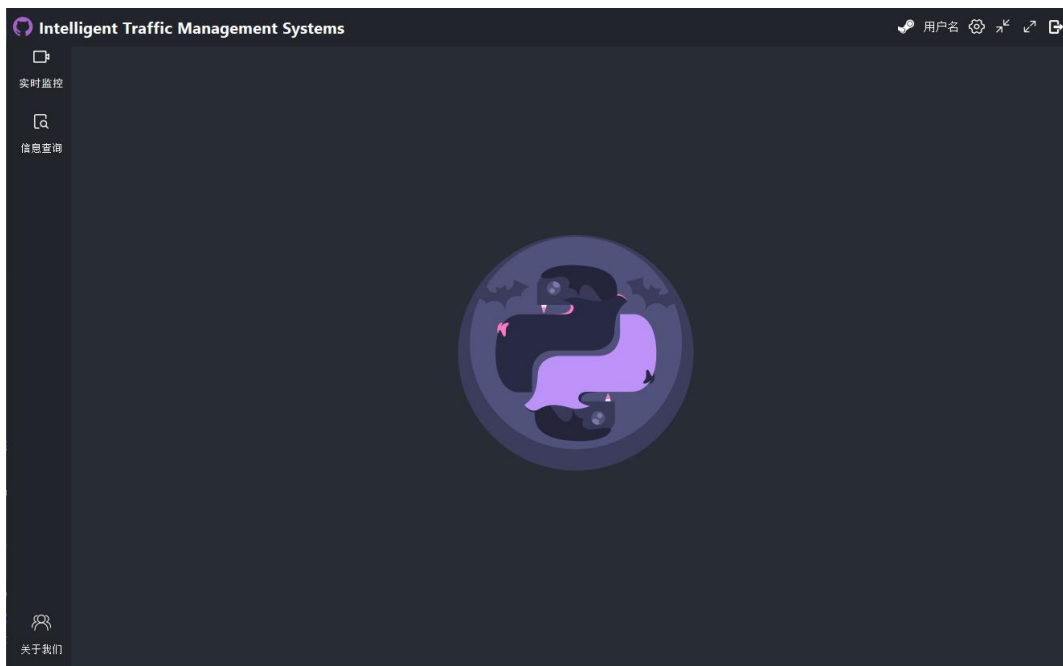


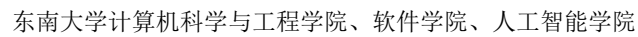
一、界面设计

1. 登录界面设计

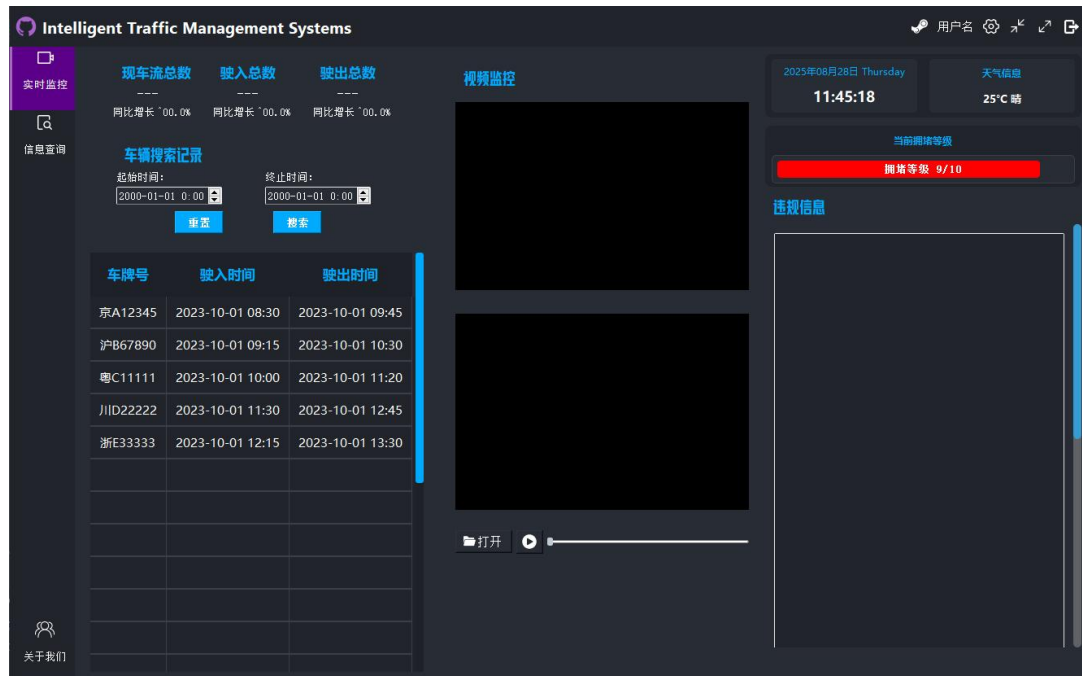


2. 主体界面设计



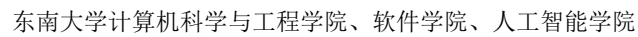


3. 实时监控界面设计



4. 信息查询界面设计





Intelligent Traffic Management Systems

实时监控

违规查询

流量查询

信息查阅

历史数据

导出数据

时间	地点	车流量	拥堵等级	查看详情
2023-10-01 08:00	中山路与解放路交叉口	120	畅通	查看
2023-10-01 09:00	南京东路	280	轻度拥堵	查看
2023-10-01 10:00	人民广场	450	中度拥堵	查看
2023-10-01 11:00	淮海中路	320	轻度拥堵	查看
2023-10-01 12:00	徐家汇路口	380	中度拥堵	查看
2023-10-01 13:00	外滩隧道	150	畅通	查看

流量数据分析

关于我们



7. 关于我们界面设计

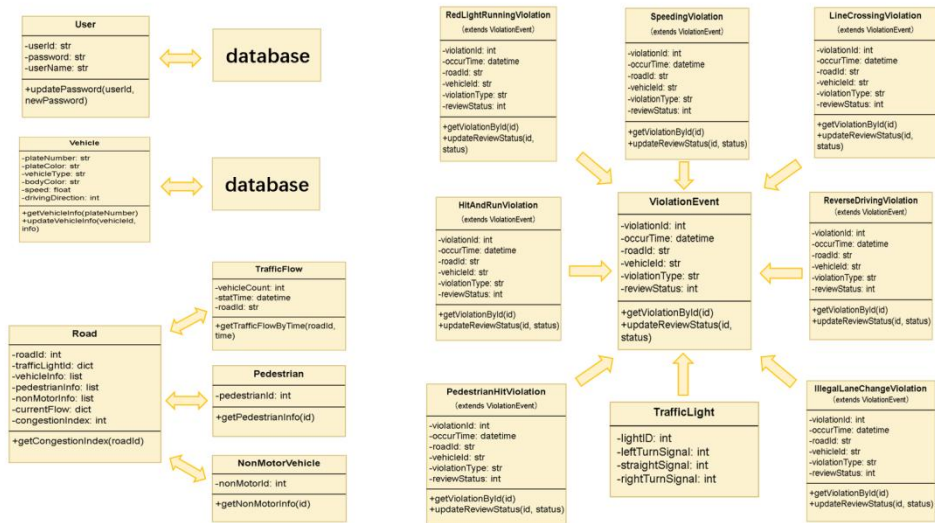


第五部分 单元模块设计

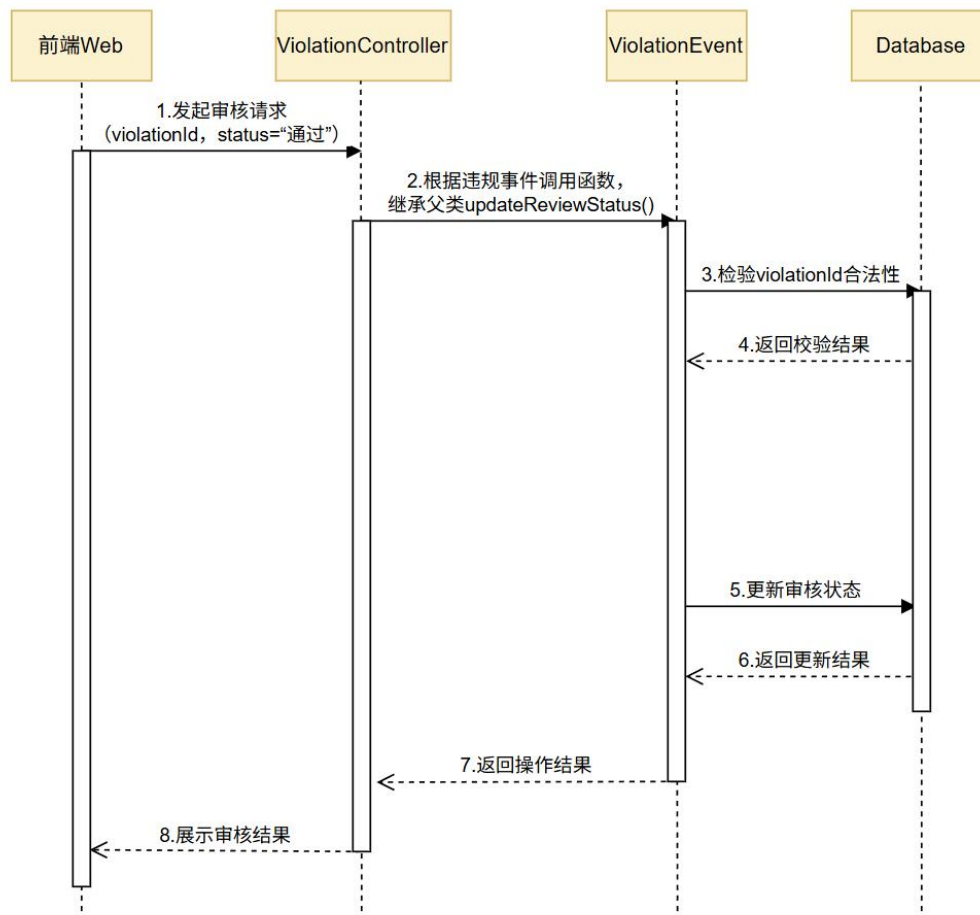
一、Model 层设计

1. 类图设计

1.1 类关系结构图



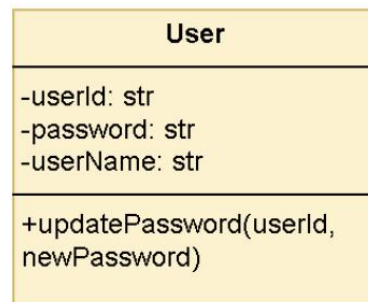
1.2 时序图





2. 类的详细设计描述

2.1 User 类



2.1.1 详细描述:

updatePassword(userId, newPassword)

接收用户 ID 和新密码，校验后更新密码并返回操作结果

2.1.2 功能实现

User 类作为用户实体模型和数据访问对象，负责用户信息的存储和密码管理：

当 AuthController 完成用户注册后，创建 User 实例并持久化到数据库，包含用户 ID、用户名和加密密码等核心属性。当用户需要修改密码时，调用 updatePassword 函数，该函数接收用户 ID 和新密码参数，先验证用户 ID 的合法性，然后对新密码进行强度校验和加密处理，最后更新数据库中的密码字段并返回操作成功结果。

2.2 Vehicle 类



Vehicle
-plateNumber: str -plateColor: str -vehicleType: str -bodyColor: str -speed: float -drivingDirection: int
+getVehicleInfo(plateNumber) +updateVehicleInfo(vehicleId, info)

2.2.1 详细描述:

getVehicleInfo(plateNumber)

根据车牌号查询并返回车辆完整信息

updateVehicleInfo(vehicleId, info)

根据车辆 ID 更新车辆信息，返回操作结果

2.2.2 功能实现

Vehicle 类作为车辆实体模型和数据访问对象，负责车辆信息的存储、查询和更新管理：

当 vehicleQueryController 或 MonitorController 需要获取或更新车辆信息时，通过 Vehicle 类提供的函数进行操作。

getVehicleInfo 函数接收车牌号参数，从数据库中查询匹配的车辆信息并返回完整的 Vehicle 对象。

updateVehicleInfo 函数接收车辆 ID 和更新信息，验证数据合法性后执行数据库更新操作。

2.3 Pedestrian 类



2.3.1 详细描述:

getPedestrianInfo(id)

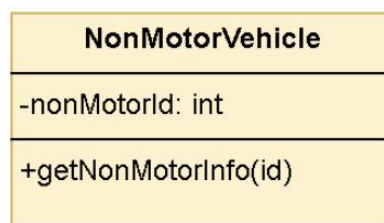
接收行人 ID，查询数据库，返回该行人的详细信息

2.3.2 功能实现

Pedestrian 类用于管理行人相关信息:

当外部系统传递行人 ID 时，调用 getPedestrianInfo 函数，先校验 ID 合法性（非负、存在性等），再查询数据库获取行人详细信息，最后返回包含行人数据的对象。

2.4 NonMotorVehicle 类



2.4.1 详细描述:

getNonMotorInfo(id)

接收非机动车 ID，查询数据库，返回该非机动车的详细信息

2.4.2 功能实现

NonMotorVehicle 类用于管理非机动车相关信息。当外部系统传递非机动车 ID 时，调用 getNonMotorInfo 函数，先校验 ID 合法性（如非负、存在性等），再查询数据库获取非



机动车详细信息，最后返回包含非机动车数据的对象。

2.5 TrafficFlow 类

TrafficFlow
-vehicleCount: int -statTime: datetime -roadId: str
+getTrafficFlowByTime(roadId, time)

2.5.1 详细描述:

getTrafficFlowByTime(roadId, time)

接收道路 ID 和时间参数，查询指定时间段内该道路的交通流量数据，返回流量统计信息

2.5.2 功能实现

TrafficFlow 类用于管理交通流量数据。当传递道路 ID 和时间参数时，调用 getTrafficFlowByTime 函数，先校验参数合法性，再查询数据库获取该时间段内的交通流量统计，最后返回流量数据对象。

2.6 Road 类



Road
-roadId: int -trafficLightId: dict -vehicleInfo: list -pedestrianInfo: list -nonMotorInfo: list -currentFlow: dict -congestionIndex: int
+getCongestionIndex(roadId)

2.6.1 详细描述:

getCongestionIndex(roadId)
接收道路 ID，综合计算该道路的拥堵指数，基于车辆信息、行人信息、非机动车信息等数据

2.6.2 功能实现

Road 类用于管理道路信息和拥堵计算，承担着道路基础设施信息维护和交通状态综合评估的重要职责。该类整合了道路相关的多维度数据，包括交通信号灯配置、各类交通参与者信息（机动车、非机动车、行人）、实时流量数据等。getCongestionIndex 函数通过复杂的算法模型，综合车辆密度、行驶速度、行人流量、非机动车数量等多重因素，计算得出 0-10 范围内的道路拥堵指数。该指数准确反映道路通行效率，为交通指挥中心提供决策依据。

2.7 TrafficLight 类

TrafficLight
-lightID: int -leftTurnSignal: int -straightSignal: int -rightTurnSignal: int



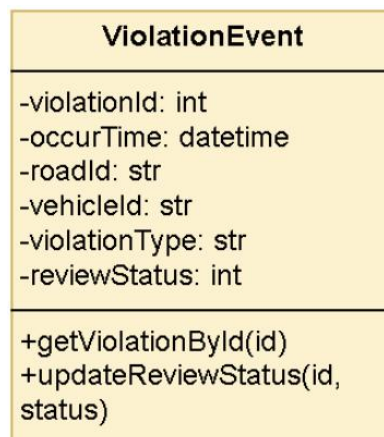
2.7.1 详细描述:

无

2.7.2 功能实现

TrafficLight 类用于表示交通信号灯实体，存储信号灯的各种信号状态，提供状态管理和查询功能。

2.8 ViolationEvent 类



2.8.1 详细描述:

getViolationById(id)

通过违规 ID 精确查询事件完整信息，支持执法取证和案件处理

updateReviewStatus(id, status)

更新违规事件的审核状态实现审核流程状态流转，确保处理过程可追溯和规范化管理

2.8.2 功能实现

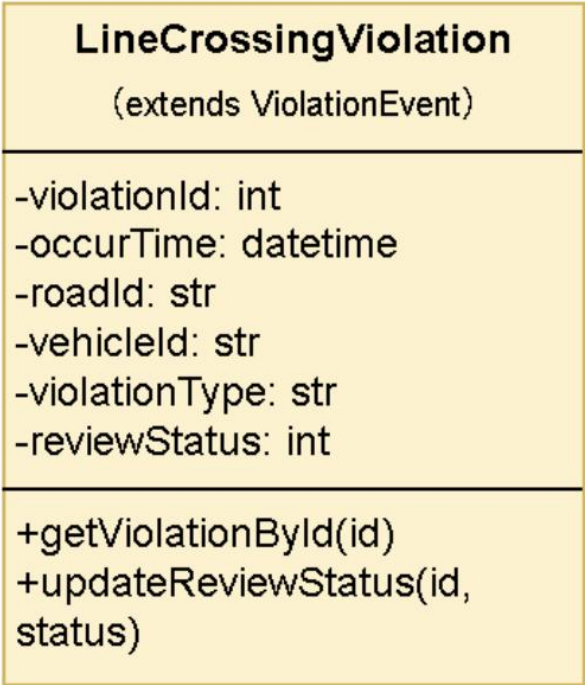
ViolationEvent 类是一个抽象类，负责交通违规事件的全程管理，包括违规信息的采集、存储、查询和审核状态维护。该类完整记录违规事件的各项要素：违规唯一标识、发生时间戳、事发道路位置、涉事车辆信息、违规类型分类以及审核处理状态。



getViolationById 函数提供精确的违规事件检索功能，确保执法部门能够快速获取违规详情。

updateReviewStatus 函数支持审核流程的状态流转，实现从待审核、审核通过到审核拒绝的全流程管理。

2.9 LineCrossingViolation 类



2.9.1 详细描述：

getViolationById(id)
通过违规 ID 精确查询事件完整信息，支持执法取证和案件处理
updateReviewStatus(id, status)
更新违规事件的审核状态实现审核流程状态流转，确保处理过程可追溯和规范化管理

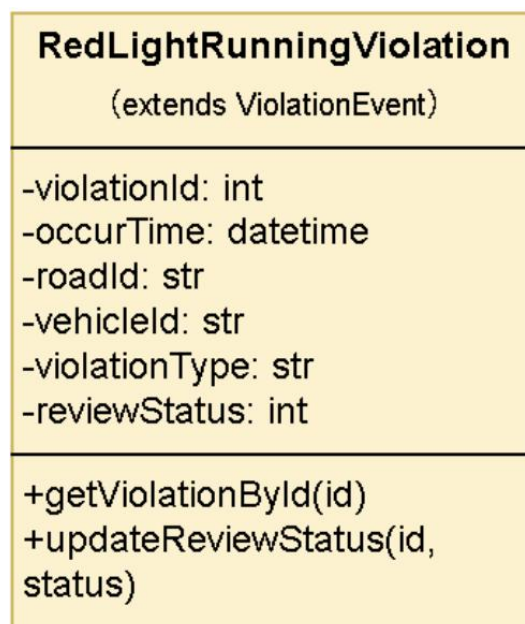
2.9.2 功能实现

LineCrossingViolation 类继承自 ViolationEvent 抽象类，定义了交通越线违规事件



相关属性，包括违规唯一标识 `violationId`、发生时间戳 `occurTime`、事发道路标识 `roadId`、涉事车辆标识 `vehicleId`、违规类型 `violationType`、审核状态 `reviewStatus`。其 `getViolationById(id)` 方法可借违规 ID 精准查询事件完整信息，辅助执法取证与案件处理；`updateReviewStatus(id, status)` 方法能更新违规事件审核状态。

2.10 RedLightRunningViolation 类



2.10.1 详细描述：

`getViolationById(id)`

通过违规 ID 精确查询事件完整信息，支持执法取证和案件处理

`updateReviewStatus(id, status)`

更新违规事件的审核状态实现审核流程状态流转，确保处理过程可追溯和规范化管理

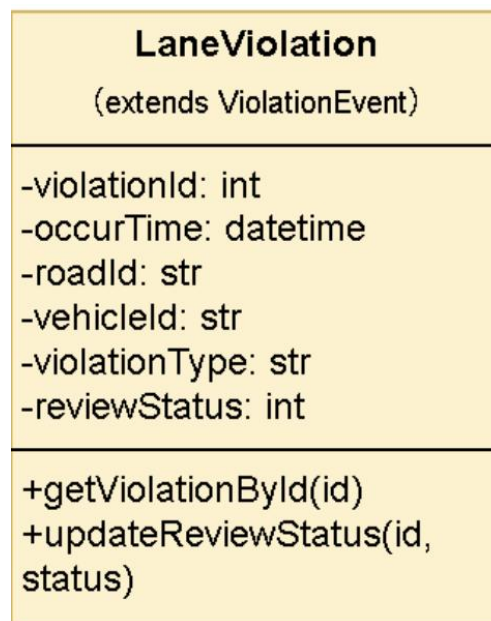
2.10.2 功能实现

`LineCrossingViolation` 类继承自 `ViolationEvent` 抽象类，定义了闯红灯违规事件相关属性，包括违规唯一标识 `violationId`、发生时间戳 `occurTime`、事发道路标识 `roadId`、



涉事车辆标识 `vehicleId`、违规类型 `violationType`、审核状态 `reviewStatus`。其 `getViolationById(id)` 方法可借违规 ID 精准查询事件完整信息,辅助执法取证与案件处理;
`updateReviewStatus(id, status)` 方法能更新违规事件审核状态。

2.11 LaneViolation 类



2.11.1 详细描述:

`getViolationById(id)`

通过违规 ID 精确查询事件完整信息,支持执法取证和案件处理

`updateReviewStatus(id, status)`

更新违规事件的审核状态实现审核流程状态流转,确保处理过程可追溯和规范化管理

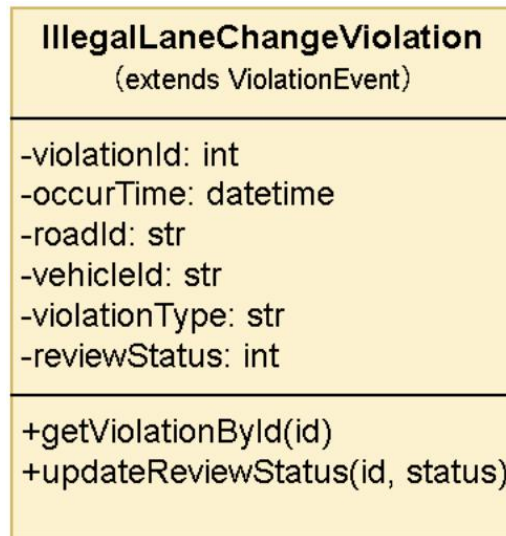
2.11.2 功能实现

`LaneViolation` 类继承自 `ViolationEvent` 抽象类,定义了不按规定车道行驶违规事件相关属性,包括违规唯一标识 `violationId`、发生时间戳 `occurTime`、事发道路标识 `roadId`、涉事车辆标识 `vehicleId`、违规类型 `violationType`、审核状态 `reviewStatus`。其



`getViolationById (id)`方法可借违规 ID 精准查询事件完整信息,辅助执法取证与案件处理;
`updateReviewStatus(id, status)`方法能更新违规事件审核状态。

2.12 IllegalLaneChangeViolation 类



2.12.1 详细描述:

`getViolationById(id)`

通过违规 ID 精确查询事件完整信息,支持执法取证和案件处理

`updateReviewStatus(id, status)`

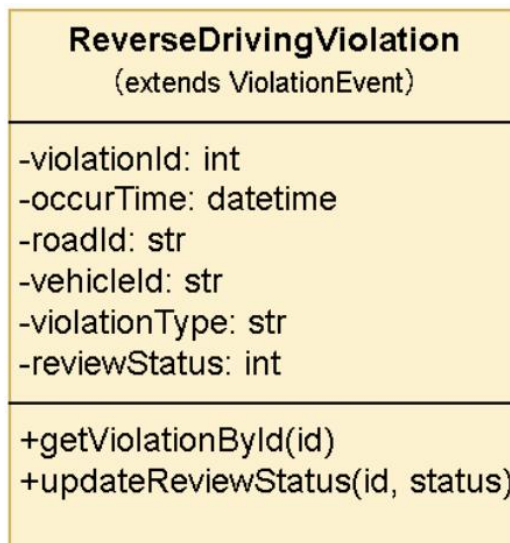
更新违规事件的审核状态实现审核流程状态流转,确保处理过程可追溯和规范化管理

2.12.2 功能实现

`IllegalLaneChangeViolation` 类继承自 `ViolationEvent` 抽象类,定义了不按规定车道行驶违规事件相关属性,包括违规唯一标识 `violationId`、发生时间戳 `occurTime`、事发道路标识 `roadId`、涉事车辆标识 `vehicleId`、违规类型 `violationType`、审核状态 `reviewStatus`。其 `getViolationById (id)`方法可借违规 ID 精准查询事件完整信息,辅助执法取证与案件处理;`updateReviewStatus(id, status)`方法能更新违规事件审核状态。



2.13 ReverseDrivingViolation 类



2.13.1 详细描述:

getViolationById(id)

通过违规 ID 精确查询事件完整信息，支持执法取证和案件处理

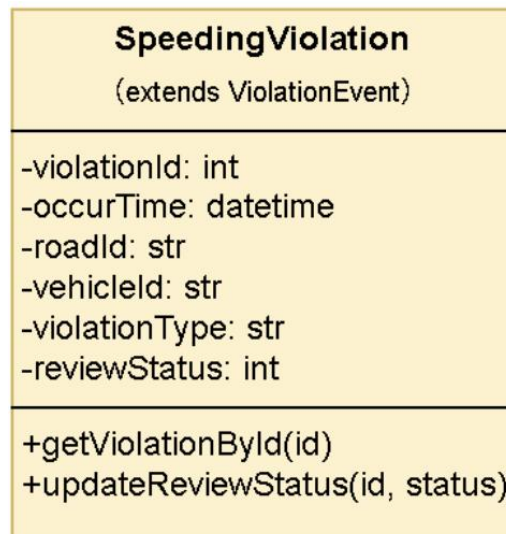
updateReviewStatus(id, status)

更新违规事件的审核状态实现审核流程状态流转，确保处理过程可追溯和规范化管理

2.13.2 功能实现

ReverseDrivingViolation 类继承自 ViolationEvent 抽象类，定义了逆向行驶事件相关属性，包括违规唯一标识 violationId、发生时间戳 occurTime、事发道路标识 roadId、涉事车辆标识 vehicleId、违规类型 violationType、审核状态 reviewStatus。其 getViolationById(id) 方法可借违规 ID 精准查询事件完整信息，辅助执法取证与案件处理；updateReviewStatus(id, status) 方法能更新违规事件审核状态。

2.14 SpeedingViolation 类



2.14.1 详细描述:

getViolationById(id)

通过违规 ID 精确查询事件完整信息，支持执法取证和案件处理

updateReviewStatus(id, status)

更新违规事件的审核状态实现审核流程状态流转，确保处理过程可追溯和规范化管理

2.14.2 功能实现

SpeedingViolation 类继承自 ViolationEvent 抽象类，定义了超速违规事件相关属性，包括违规唯一标识 violationId、发生时间戳 occurTime、事发道路标识 roadId、涉事车辆标识 vehicleId、违规类型 violationType、审核状态 reviewStatus。其 getViolationById(id) 方法可借违规 ID 精准查询事件完整信息，辅助执法取证与案件处理；updateReviewStatus(id, status) 方法能更新违规事件审核状态。

2.15 HitAndRunViolation 类



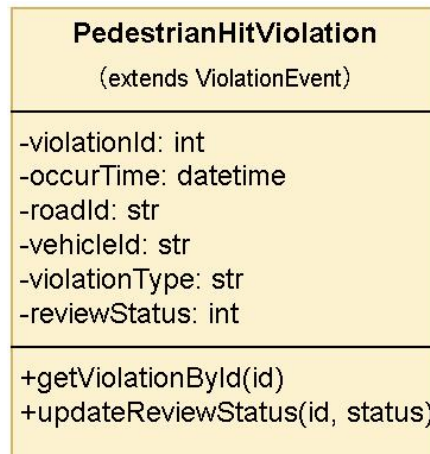
2.15.1 详细描述:

getViolationById(id)
通过违规 ID 精确查询事件完整信息，支持执法取证和案件处理
updateReviewStatus(id, status)
更新违规事件的审核状态实现审核流程状态流转，确保处理过程可追溯和规范化管理

2.15.2 功能实现

HitAndRunViolation 类继承自 ViolationEvent 抽象类，定义了肇事逃逸违规事件相关属性，包括违规唯一标识 violationId、发生时间戳 occurTime、事发道路标识 roadId、涉事车辆标识 vehicleId、违规类型 violationType、审核状态 reviewStatus。其 getViolationById(id) 方法可借违规 ID 精准查询事件完整信息，辅助执法取证与案件处理；updateReviewStatus(id, status) 方法能更新违规事件审核状态。

2.16 PedestrianHitViolation 类



2.16.1 详细描述:

getViolationById(id)

通过违规 ID 精确查询事件完整信息，支持执法取证和案件处理

updateReviewStatus(id, status)

更新违规事件的审核状态实现审核流程状态流转，确保处理过程可追溯和规范化管理

2.16.2 功能实现

PedestrianHitViolation 类继承自 ViolationEvent 抽象类,定义了肇事伤人相关属性,包括违规唯一标识 violationId、发生时间戳 occurTime、事发道路标识 roadId、涉事车辆标识 vehicleId、违规类型 violationType、审核状态 reviewStatus。其 getViolationById(id) 方法可借违规 ID 精准查询事件完整信息，辅助执法取证与案件处理；updateReviewStatus(id, status) 方法能更新违规事件审核状态。

二、Service 层设计

1. 类图设计

1.1 类关系结构图



UserService +createUser(user: User) +validateLogin(userName, password) +updatePassword(userId, newPassword)	ViolationService +fetchViolations(condition) +getEvidence(violationId) +updateViolationRecord(violationId, data) +reviewViolation(violationId, status) +addViolation(violationId, data)	TrafficFlowService +fetchRealtimeFlow(roadId) +fetchHistoryFlow(roadId, timeRange) +analyzeFlowTrend(roadId) +calculateCongestionIndex(roadId)	AIAnalysisService +analyzeVideoStream(streamUrl) +detectVehicles(frame) +recognizePlate(frame) +detectViolation(frame) +sendAlert(event)
---	---	---	--

2. 类的详细设计描述

2.1 UserService 类

UserService
+createUser(user: User) +validateLogin(userName, password) +updatePassword(userId, newPassword)

2.1.1 详细描述:

createUser(user)
创建新用户账户，接收用户对象参数，完成数据校验后持久化存储到数据库系统
validateLogin(userName, password)
验证用户登录凭证，比对用户名密码匹配性，返回认证结果和用户状态信息
updatePassword(userId, newPassword)
更新用户密码，验证身份后加密存储新密码，确保账户安全性和密码强度合规

2.1.2 功能实现

UserService 类负责用户账户的管理，处理用户创建、登录验证和密码更新等核心业务逻辑。

当接收到用户创建请求时，调用 createUser 函数进行数据完整性校验，确保用户名唯一性和密码复杂度要求，通过后调用数据访问层完成用户信息持久化存储。

在用户登录验证场景中，validateLogin 函数比对输入凭证与数据库记录，验证账户状



态是否正常，返回包含用户 ID 和权限的认证结果。

密码更新功能通过 `updatePassword` 函数实现，首先验证用户身份合法性，然后对新密码进行加密处理，最终更新数据库记录并返回操作结果。

2.2 ViolationService 类

ViolationService
+fetchViolations(condition) +getEvidence(violationId) +updateViolationRecord(violationId, data) +reviewViolation(violationId, status) +addViolation(violationId, data)

2.2.1 详细描述:

fetchViolations(condition)
根据查询条件检索违规记录，支持多条件组合筛选，返回分页结果集
getEvidence(violationId)
获取指定违规 ID 的证据材料，包括图片、视频等多媒体证据文件数据
updateViolationRecord(violationId, data)
更新违规记录信息，支持部分字段修改，记录操作日志备查
reviewViolation(violationId, status)
审核违规记录，更新审核状态并添加审核意见，完成处理流程
addViolation(violationId, data)
添加新的违规记录，接收违规详细信息参数，完成数据校验后入库存储

2.2.2 功能实现

`ViolationService` 类承担交通违规数据的管理职责，提供违规记录的新增、查询、更新和审核等完整业务流程支持。

`fetchViolations` 函数根据传入的查询条件动态构建检索语句，返回符合条件的分页违规数据，支持按时间、地点、类型等多维度筛选。



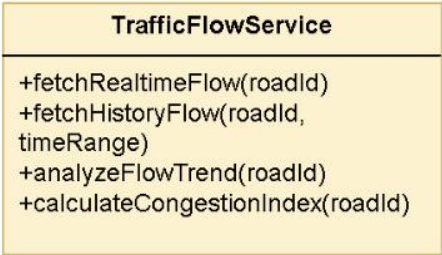
getEvidence 函数通过违规 ID 关联获取相应的证据材料，包括违规瞬间的图片抓拍和视频录像等多媒体证据。

updateViolationRecord 函数允许对已有违规记录进行信息修正。

reviewViolation 函数专门处理审核状态流转。

addViolation 函数负责接收前端采集的新增违规数据，经过严格校验后持久化存储到数据库系统中。

2.3 TrafficFlowService 类



2.3.1 详细描述:

fetchRealtimeFlow(roadId)
获取道路实时流量数据，包含当前车辆数量、平均速度等实时统计信息
fetchHistoryFlow(roadId, timeRange)
查询历史流量数据，支持按时间范围检索，返回时序流量统计数据
analyzeFlowTrend(roadId)
分析流量变化趋势，识别高峰时段和流量模式，提供趋势预测分析报告
calculateCongestionIndex(roadId)
计算道路拥堵指数，综合多种因素评估道路通行状态，生成拥堵评级

2.3.2 功能实现

TrafficFlowService 类负责实时和历史交通流量数据的采集、存储与分析，通过计算



拥堵指数和识别流量趋势，为交通管理和道路规划提供数据支撑与决策依据。

`fetchRealtimeFlow` 函数从实时数据源获取当前道路流量信息，包括车辆计数、速度分布等关键指标。

`fetchHistoryFlow` 函数则从历史数据库中检索指定时间范围内的流量数据，支持时序对比分析。

`analyzeFlowTrend` 函数运用统计算法识别流量变化规律和趋势特征，为交通规划提供参考。

`calculateCongestionIndex` 函数综合实时流量、历史数据和道路容量等因素，通过复杂算法计算出客观的拥堵指数，准确反映道路通行状况。

2.4 AIAnalysisService 类

AIAnalysisService
+analyzeVideoStream(streamUrl) +detectVehicles(frame) +recognizePlate(frame) +detectViolation(frame) +sendAlert(event)

2.4.1 详细描述:

analyzeVideoStream(streamUrl)
分析实时视频流数据，持续处理视频帧并进行交通事件检测分析
detectVehicles(frame)
检测视频帧中的车辆目标，识别车辆位置、大小和类型等视觉特征信息
recognizePlate(frame)
识别车辆牌照信息，提取牌照号码、颜色和地域等关键识别特征数据
detectViolation(frame)
检测交通违规行为，分析车辆运动轨迹并识别各类违规模式特征
sendAlert(event)



发送事件警报通知，将检测到的违规事件实时推送到相关处理人员系统

2.4.2 功能实现

AIAnalysisService 类集成了人工智能视觉分析技术，实现对交通场景的智能监控和事件自动识别。

analyzeVideoStream 函数持续处理输入的视频流数据，逐帧调用 detectVehicles 函数进行车辆目标检测，准确识别车辆在画面中的位置和类型特征。

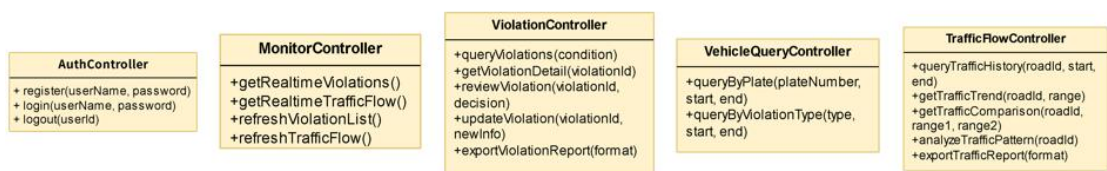
recognizePlate 函数进一步提取检测到的车辆目标的牌照信息，完成车辆身份识别。

detectViolation 函数通过分析车辆运动轨迹和行为模式，自动识别闯红灯、压线、超速等各种交通违规行为。

一旦发现违规事件，sendAlert 函数立即生成警报通知，将事件详细信息实时推送到交通管理平台，实现从视频分析到事件处置的完整闭环处理流程。

三、Controller 层设计

1. 类图设计



2. 类的详细描述

2.1 AuthController 类



AuthController

```
+ register(userName, password)
+ login(userName, password)
+ logout(userId)
```

2.1.1 详细描述:

register(userName, password)

接收 View 层账号密码，校验后入库并返回注册结果，含用户 id

login(userName, password)

接收登录请求，校验账号密码与状态，通过则初始化会话并返回结果

logout(userId)

接收登出请求，校验后记录日志、销毁会话，返回结果

2.1.2 功能实现

AuthController 类依据 View 层传来的用户请求，调用自身对应函数完成认证流程：当 View 层传递注册所需的账号密码时，调用 register 函数，先校验参数合规性，再完成用户信息入库，最后返回含用户 id 的注册结果；当接收到登录请求与账号密码参数时，调用 login 函数，校验账号密码正确性与用户状态，通过后初始化会话并返回登录结果；当 View 层传递用户 id 发起登出请求时，调用 logout 函数，校验请求合法性后记录登出日志、销毁会话，返回登出结果，实现不同认证场景的流程闭环。

2.2 MonitorController 类



MonitorController

```
+getRealtimeViolations()  
+getRealtimeTrafficFlow()  
+refreshViolationList()  
+refreshTrafficFlow()
```

2.2.1 详细描述:

`getRealtimeViolations()`

接收 View 层请求，获取并返回实时交通违规数据。

`getRealtimeTrafficFlow()`

接收 View 层请求，获取并返回实时交通流量数据。

`refreshViolationList()`

响应用户刷新操作，重新获取违规数据并更新列表返回。

`refreshTrafficFlow()`

响应用户刷新操作，重新获取流量数据并更新返回。

2.2.2 功能实现

MonitorController 类根据 View 层不同数据获取、刷新请求，调用对应函数：收到实时违规数据需求，`getRealtimeViolations` 从数据源取数返回；需实时流量数据，`getRealtimeTrafficFlow` 联动服务采集加工回传；用户主动刷新违规列表/流量时，`refreshViolationList`、`refreshTrafficFlow` 触发数据重获取更新，保障监控数据动态展示。

2.3 ViolationController 类



ViolationController

```
+queryViolations(condition)
+getViolationDetail(violationId)
+reviewViolation(violationId,
decision)
+updateViolation(violationId,
newInfo)
+exportViolationReport(format)
```

2.3.1 详细描述:

queryByPlate(plateNumber, start, end)

接收车牌、时间范围，查询对应车辆指定时段交通记录并返回。

queryByViolationType(type, start, end)

接收违规类型、时间范围，查询对应类型时段违规记录并返回。

reviewViolation(violationId, decision)

接收违规 ID 和审核结果，执行审核流程并更新状态

updateViolation(violationId, newInfo)

接收违规 ID 和更新信息，编辑违规记录并反馈结果。

exportViolationReport(format)

接收报告格式，生成并返回对应格式的违规报表。

2.3.2 功能实现

ViolationController 类依据 View 层发起的不同业务动作（查询、详情查看、审核、编辑、导出），调用对应函数处理：收到带条件的违规查询请求，queryViolations 调用违规数据服务，按条件从数据库等数据源筛选数据回传；查看某条违规详情，getViolationDetail 调用数据查询逻辑，提取完整违规信息响应；执行违规审核操作时，reviewViolation 调用业务流程服务，更新审核状态并反馈结果；需违规记录编辑需求时，



updateViolation 触发数据更新逻辑，校验并修改违规信息；若导出违规报告，exportViolationReport 调用报表生成服务，按指定格式封装数据并返回。

2.4 VehicleQueryController 类

VehicleQueryController

```
+queryByPlate(plateNumber,  
start, end)  
+queryByViolationType(type,  
start, end)
```

2.4.1 详细描述:

queryByPlate(plateNumber, start, end)

接收车牌、时间范围，查询对应车辆指定时段交通记录并返回。

queryByViolationType(type, start, end)

接收违规类型、时间范围，查询对应类型时段违规记录并返回。

2.4.2 功能实现

VehicleQueryController 类依据 View 层不同车辆及违规查询需求，调用对应函数处理：当收到按车牌号查询的请求，queryByPlate 调用数据服务，从数据库等数据源中，按车牌号和时间范围检索匹配记录并回传；若需按违规类型查询，queryByViolationType 调用数据筛选逻辑，提取指定违规类型、时间区间内的违规数据响应。

2.5 TrafficFlowController 类



TrafficFlowController

```
+queryTrafficHistory(roadId, start, end)
+getTrafficTrend(roadId, range)
+getTrafficComparison(roadId, range1, range2)
+analyzeTrafficPattern(roadId)
+exportTrafficReport(format)
```

2.5.1 详细描述:

queryTrafficHistory(roadId, start, end)

接收道路 ID 与时间区间，查询并返回对应道路指定时段的历史交通流量数据。

getTrafficTrend(roadId, range)

接收道路 ID 与时间范围，分析并返回该道路交通流量的变化趋势数据。

getTrafficComparison(roadId, range1, range2)

接收道路 ID 与两个时间范围，对比并返回道路在不同时段的交通流量差异数据。

analyzeTrafficPattern(roadId)

接收道路 ID，挖掘并返回对应道路的交通流量模式与特征。

exportTrafficReport(format)

接收报告格式，生成并返回含流量数据、趋势等内容的交通流量分析报告。

2.5.2 功能实现

TrafficFlowController 类依据 View 层对交通流量的查询、分析、导出等需求，调用对应函数处理：收到历史流量查询，queryTrafficHistory 联动数据服务，从存储中按道路和时间筛数据回传；需趋势分析，getTrafficTrend 调用分析逻辑，加工流量数据生成趋势结果；做流量对比时，getTrafficComparison 提取两时段数据计算差异；执行模式挖掘，analyzeTrafficPattern 运用算法识别流量规律；要导出报告，exportTrafficReport 协同报表服务，按格式封装内容并返回。



四、View 层设计

1. 界面功能

本系统的 View 层主要负责用户界面展示与交互，涵盖登录、主界面、实时监控、违法查询与审核、统计分析、交通流量等功能。功能模块汇总如下：

模块名称	功能描述
登录界面 (LoginView)	用户输入账号和密码，进行登录或注册操作，并反馈登录状态。
主界面 (MainDashboardView)	提供菜单栏与状态栏，支持在实时监控、信息查询等模块之间切换。
实时违法监控 (RealtimeViolationView)	展示违法行为列表，实时刷新数据，并提供视频监控流播放功能。
实时交通流量监控 (RealtimeTrafficFlowView)	显示交通流量图表与拥堵指数，可实时更新。
违法查询 (ViolationQueryView)	输入条件查询违法记录，并展示查询结果。
违法审核 (ViolationReviewView)	显示违法详情与证据，支持审核通过、驳回、修改操作。
车牌号查询 (PlateQueryView)	根据车牌号与时间范围查询相关违法记录。
违法类型查询 (ViolationTypeQueryView)	根据违法类型和时间范围查询违法记录。
违法统计 (ViolationStatisticsView)	展示违法趋势图、热力图、类型饼图，并支持导出统计报表。
交通流量查询 (TrafficFlowQueryView)	根据时间范围查询交通流量数据并展示结果。
交通流量统计 (TrafficFlowStatisticsView)	展示交通流量的趋势、占比、对比等图表，支持模式分析与导出报表。

2. 类的详细描述

2.1 LoginView 类



LoginView
-userNameInput -passwordInput -loginButton -registerButton -statusLabel
+initUI() +onLoginClicked() +onRegisterClicked() +showMessage(msg)

initUI() 初始化登录界面，加载输入框、按钮和状态提示控件。
onLoginClicked() 处理登录按钮点击事件，获取用户输入并触发登录逻辑。
onRegisterClicked() 处理注册按钮点击事件，跳转到注册逻辑。
showMessage(msg) 接收提示信息并显示在状态栏上。

2.2 MainDashboardView 类

MainDashboardView
-menuBar -statusBar -monitorPanel -queryPanel
+initUI() +switchToMonitorView() +switchToQueryView()

initUI() 初始化主界面，加载菜单栏、状态栏及面板。
switchToMonitorView() 切换到实时监控界面。
switchToQueryView() 切换到信息查询界面。



2.3 RealtimeViolationView 类

RealtimeViolationView
-violationTable -videoPanel
+showViolationList(data) +refreshViolationList() +showViolationVideo(stream)

showViolationList(data) 接收违法数据并填充到违法表格中。
refreshViolationList() 刷新违法表格，更新最新的违法信息。
showViolationVideo(stream) 播放违法行为对应的视频流。

2.4 RealtimeTrafficFlowView 类

RealtimeViolationView
-violationTable -videoPanel
+showViolationList(data) +refreshViolationList() +showViolationVideo(stream)

showFlowData(data) 接收并展示实时交通流量数据。
updateCongestionIndex(value) 更新并显示道路的拥堵指数。

2.5 ViolationQueryView 类



ViolationQueryView
-queryInput -queryButton -violationTable
+onQueryClicked() +showQueryResult(data)

onQueryClicked()

触发违法记录查询操作。

showQueryResult(data)

接收查询结果并显示在表格中。

2.6 ViolationReviewView 类

ViolationReviewView
-violationDetailPanel -evidencePanel -approveButton -rejectButton -modifyButton
+showViolationDetail(data) +onApproveClicked() +onRejectClicked(reason) +onModifyClicked(data)

showViolationDetail(data)

展示违法详情信息及证据。

onApproveClicked()

审核通过，确认违法记录有效。

onRejectClicked(reason)

驳回违法记录，并记录驳回原因。

onModifyClicked(data)

修改违法记录或证据信息。

2.7 PlateQueryView 类



PlateQueryView
-plateInput -startTimePicker -endTimePicker -resultTable
+onQueryClicked() +showResult(data)

onQueryClicked()

触发车牌号查询操作。

showResult(data)

接收查询结果并展示在结果表格中。

2.8 ViolationTypeQueryView 类

ViolationTypeQueryView
-typeDropdown -startTimePicker -endTimePicker -resultTable
+onQueryClicked() +showResult(data)

onQueryClicked()

触发按违法类型的查询操作。

showResult(data)

接收查询结果并展示在结果表格中。

2.9 ViolationStatisticsView 类

ViolationStatisticsView
-trendChart -heatMap -typePieChart -exportButton
+showStatistics(data) +exportReport(format)



showStatistics(data)

接收统计数据并展示趋势图、热力图、饼图等。

exportReport(format)

接收导出格式，生成并导出违法统计报表。

2.10 TrafficFlowQueryView 类

TrafficFlowQueryView

-startPicker
-endPicker
-resultTable

+onQueryClicked()
+showResult(data)

onQueryClicked()

触发交通流量查询操作。

showResult(data)

接收查询结果并展示在表格中。

2.11 TrafficFlowStatisticsView 类

TrafficFlowStatisticsView

-trendChart
-pieChart
-barChart
-comparisonChart

+showStatistics(data)
+analyzePattern(data)
+exportReport(format)

showStatistics(data)

接收交通流量统计数据并展示趋势图、饼图、柱状图、对比图等。

analyzePattern(data)

分析并挖掘交通流量模式与规律。

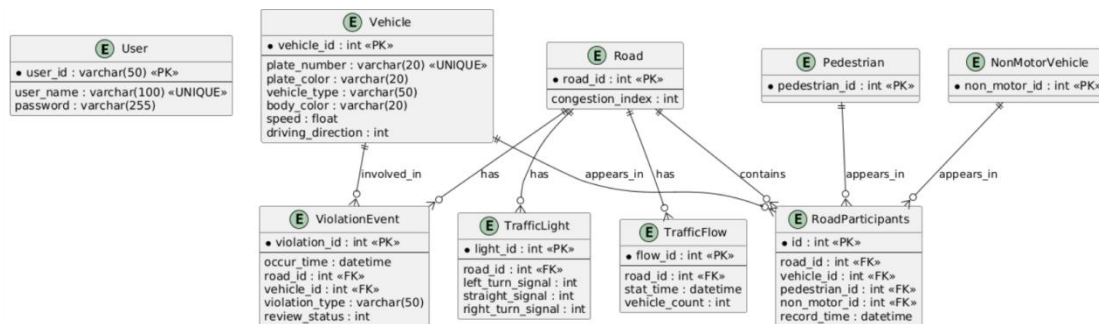


exportReport(format)

接收导出格式，生成并导出流量统计报表。

第六部分 数据库设计

一、数据库整体结构图



二、数据库表设计

1. 用户表 (User)

字段名	类型	约束/说明
user_id	VARCHAR(50)	PRIMARY KEY
user_name	VARCHAR(100)	NOT NULL UNIQUE
password	VARCHAR(255)	NOT NULL

2. 车辆表 (Vehicle)

字段名	类型	约束/说明
-----	----	-------



vehicle_id	SERIAL	PRIMARY KEY
plate_number	VARCHAR(20)	NOT NULL UNIQUE
plate_color	VARCHAR(20)	
vehicle_type	VARCHAR(50)	
body_color	VARCHAR(20)	
speed	FLOAT	
driving_direction	INT	

3. 行人表 (Pedestrian)

字段名	类型	约束/说明
pedestrian_id	SERIAL	PRIMARY KEY

4. 非机动车表 (NonMotorVehicle)

字段名	类型	约束/说明
non_motor_id	SERIAL	PRIMARY KEY

5. 道路表 (Road)

字段名	类型	约束/说明
-----	----	-------



road_id	SERIAL	PRIMARY KEY
congestion_index	INT	

6. 信号灯表 (TrafficLight)

字段名	类型	约束/说明
light_id	SERIAL	PRIMARY KEY
road_id	INT	REFERENCES Road(road_id) ON DELETE CASCADE
left_turn_signal	INT	
straight_signal	INT	
right_turn_signal	INT	

7. 交通流量表 (TrafficFlow)

字段名	类型	约束/说明
flow_id	SERIAL	PRIMARY KEY
vehicle_count	INT	
stat_time	TIMESTAMP	NOT NULL
road_id	INT	REFERENCES Road(road_id) ON DELETE CASCADE



8. 违章事件基表 (ViolationEvent)

字段名	类型	约束/说明
violation_id	SERIAL	PRIMARY KEY
occur_time	TIMESTAMP	NOT NULL
road_id	INT	REFERENCES Road(road_id) ON DELETE SET NULL
vehicle_id	INT	REFERENCES Vehicle(vehicle_id) ON DELETE SET NULL
violation_type	VARCHAR(50)	NOT NULL ('LineCrossing', 'RedLightRunning', ...)
review_status	INT	DEFAULT 0 (0=未审核, 1= 通过, 2=驳回)

第七部分 补充设计和说明

一、编译运行环境设计

1. 基础环境要求

操作系统：开发环境（Windows 10/11、macOS 12+）；生产环境（Linux CentOS 7+/Ubuntu 20.04+）

Python 版本：3.8.x（64 位）



GPU 支持：NVIDIA GPU（算力 ≥ 6.0 ），需配合 CUDA 11.1 和 cuDNN 8.0.5

2. 核心依赖说明

基础工具：requests==2.32.4、urllib3==2.2.3 等（网络请求与编码支持）

数据处理：numpy==1.24.4、pandas==2.0.3（数值计算与表格处理）

可视化与 UI：matplotlib==3.7.5、PyQt5==5.15.11（绘图与界面交互）

深度学习：torch==1.8.1+cu111、torchvision==0.9.1+cu111（模型训练推理）

机器学习：scikit-learn==1.3.0、scipy==1.10.0（经典算法支持）

数据库与配置：sqlalchemy==2.0.43、PyYAML==6.0.2（数据存储与配置解析）

3. 部署步骤

- 1) 安装 Python 3.9.x 并配置环境变量
- 2) （GPU 环境）安装 CUDA 11.1 和 cuDNN 8.0.5
- 3) 创建 requirements.txt 文件（包含所有指定依赖）
- 4) 执行安装命令：

```
pip install torch==1.8.1+cu111 torchvision==0.9.1+cu111 torchaudio==0.8.1
-f https://download.pytorch.org/whl/torch_stable.html

pip install -r requirements.txt
```



4. 验证与问题排查

关键验证：`torch.cuda.is_available()`（GPU 环境应返回 `True`）、PyQt5 窗口测试、
数据处理功能测试

常见问题：PyQt5 需安装 `PyQt5-tools`；CUDA 版本不匹配会导致 `torch` 无法使用 GPU

5. 维护说明

依赖包版本原则上不更新，确需更新需先在测试环境验证

定期备份依赖清单：`pip freeze > requirements.txt`